Project 3: Writing your Application

Group 27
Steven Chang, Eleni Lyberopoulos, Alice McMurray, Mujtaba Mirhasan

Relational Schema:

Product(product_id, description, name, amount_in_stock, price)

Toy(product_id, material, age_group, dimensions, manufacturer)

- product_id is a foreign key to Product

Movie(product_id, rating, duration, director, release_date)

- product_id is a foreign key to Product

Book(product_id, published_date, publisher, ISBN, page_count)

- product_id is a foreign key to Product

Order(order_id, order_timestamp, customer_id, payment_id, shipping_id)

- customer_id is a foreign key to Customer
- payment_id is a foreign key to Payment
- shipping_id is a foreign key to Shipping

Payment(payment_id, payment_timestamp, payment_method, transaction_token, amount, last_four_digits)

Shipping(shipping_id, from_address, to_address, tracking_number, carrier_name, status, expected_delivery)

Customer(customer_id, first_name, last_name, email, password)

Author(author_id, author_name, biography)

Genre(genre_name)

Return(return_timestamp, order_id, product_id, quantity, shipping_id)

- (order_id, product_id) is a foreign key to OrderItem
- shipping_id is a foreign key to Shipping

OrderItem(order_id, product_id, quantity, price_at_purchase)

- order_id is a foreign key to Order
- product_id is a foreign key to Product

BookAuthor(author_id, product_id)

- author_id is a foreign key to Author
- product_id is a foreign key to Product

BookGenre(product_id, genre_name)

- product_id is a foreign key to Book
- genre_name is a foreign key to Genre

MovieGenre(product_id, genre_name)

- product_id is a foreign key to Movie
- genre_name is a foreign key to Genre

Stored Procedure:

a) The procedure CalculateBookRevenueAfterTax takes in two inputs chosen by the user: a year and a tax rate. The program:
   1. Drops and recreates a table called RevenueAfterTax to store the results.
   2. It loops through all book sales in the given year.
   3. For each book sale, it finds all authors of that book.
   4. It calculates the gross revenue (price*quantity) and then applies the tax rate to compute net revenue.
   5. If a book has multiple authors, the revenue is split evenly among them.
   6. Each author's share of revenue is inserted into the RevenueAfterTax table.
   7. The procedure also keeps track of the total net revenue across all books and authors and returns this total at the end.

b) Code for the stored procedure:

```
CONNECT TO COMP421@

CREATE PROCEDURE CalculateBookRevenueAfterTax(
        IN inputYear INT,
        IN inputTaxRate DECIMAL(5, 2),
        OUT totalNet DECIMAL(15, 2)
)
LANGUAGE SQL
BEGIN
        -- Variables
        DECLARE v_done INT DEFAULT 0; -- boolean to track if cursor has reached the end
                                      -- (rows returned by SELECT query)

        DECLARE v_order_id INT;
        DECLARE v_book_id INT;
        DECLARE v_price DECIMAL(10, 2);
        DECLARE v_qty INT;

        DECLARE v_author_id INT;
        DECLARE v_author_count INT DEFAULT 1;

        DECLARE v_gross DECIMAL(10, 2);
        DECLARE v_net DECIMAL(10, 2);
        DECLARE v_total_net DECIMAL(15, 2) DEFAULT 0;


        -- Cursors
        -- All book order items for the given year (inputYear)
        DECLARE c1 CURSOR FOR
                SELECT oi.order_id, oi.product_id, oi.price_at_purchase, oi.quantity,
                        ba.author_id,
                        (SELECT COUNT(*) FROM BookAuthor WHERE product_id =
oi.product_id)
                        AS author_count
                FROM OrderItem oi
```

```
                JOIN Book b ON oi.product_id = b.product_id
                JOIN "Order" o ON oi.order_id = o.order_id
                JOIN BookAuthor ba ON oi.product_id = ba.product_id
                WHERE YEAR(o.order_timestamp) = inputYear;


        -- Handler for cursor end
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;


        -- Handler to ignore error: '42704' = table does not exist'
       DECLARE CONTINUE HANDLER FOR SQLSTATE '42704' BEGIN END;


        -- Drop table if exists
        BEGIN
                DROP TABLE RevenueAfterTax;
        END;

        -- Create the table
        CREATE TABLE RevenueAfterTax (
                revenue_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                year INT NOT NULL,
                author_id INT NOT NULL,
                book_id INT NOT NULL,
                gross_revenue DECIMAL(10, 2) NOT NULL,
                tax_rate DECIMAL(5, 2) NOT NULL,
                net_revenue DECIMAL(10, 2) NOT NULL
        );

        SET totalNet = 0;

        OPEN c1;
        FETCH c1 INTO v_order_id, v_book_id, v_price, v_qty, v_author_id,
v_author_count;
        WHILE v_done = 0 DO
                SET v_gross = (v_price * v_qty) / v_author_count;
                SET v_net = v_gross * (1 - inputTaxRate);

                INSERT INTO RevenueAfterTax(year, author_id, book_id, gross_revenue,
                        tax_rate, net_revenue)
                VALUES(inputYear, v_author_id, v_book_id, v_gross, inputTaxRate,
v_net);

                SET v_total_net = v_total_net + v_net;

                -- Fetch next row
                FETCH c1 INTO v_order_id, v_book_id, v_price, v_qty, v_author_id,
                        v_author_count;
        END WHILE;

        CLOSE c1;

        -- Return total net revenue via OUT param
        SET totalNet = v_total_net;

END@
```

c) Before running the procedure:

```
db2 => SELECT * FROM RevenueAfterTax
SQL0204N  "CS421G27.REVENUEAFTERTAX" is an undefined name.  SQLSTATE=42704
```

Run the command to create the procedure:

```
cs421g27@winter2026-comp421:~/project2$ db2 -tvf project3_procedure.sql -td@
```

```
END
DB20000I  The SQL command completed successfully.
```

Call the procedure:

```
db2 => CALL CalculateBookRevenueAfterTax(2025, 0.05, ?);

  Value of output parameters
  --------------------------
  Parameter Name  : TOTALNET
  Parameter Value : 456.51

  Return Status = 0
```

d) After running the procedure:

```
db2 => SELECT * FROM RevenueAfterTax

REVENUE_ID  YEAR        AUTHOR_ID   BOOK_ID     GROSS_REVENUE TAX_RATE NET_REVENUE
----------- ----------- ----------- ----------- ------------- -------- -----------
          1        2025           1           2         19.99     0.05       18.99
          2        2025           2           5         12.49     0.05       11.86
          3        2025           8           5         12.49     0.05       11.86
          4        2025           3           8         33.50     0.05       31.82
          5        2025           6          10         21.50     0.05       20.42
          6        2025           5          14         18.75     0.05       17.81
          7        2025           4          17         22.99     0.05       21.84
          8        2025           5          19          8.49     0.05        8.06
          9        2025           7          19          8.49     0.05        8.06
         10        2025           5          24         20.00     0.05       19.00
         11        2025           1           2         19.99     0.05       18.99
         12        2025           2           5         24.99     0.05       23.74
         13        2025           8           5         24.99     0.05       23.74
         14        2025           3           8         14.25     0.05       13.53
         15        2025           6          10         21.50     0.05       20.42
         16        2025           5          14         37.50     0.05       35.62
         17        2025           4          17         22.99     0.05       21.84
         18        2025           5          19          8.49     0.05        8.06
         19        2025           7          19          8.49     0.05        8.06
         20        2025           5          24         20.00     0.05       19.00
         21        2025           1           2         19.99     0.05       18.99
         22        2025           2           5          9.00     0.05        8.55
         23        2025           8           5          9.00     0.05        8.55
         24        2025           3           8         14.25     0.05       13.53
         25        2025           6          10         21.50     0.05       20.42
         26        2025           6          10         25.00     0.05       23.75

  26 record(s) selected.
```

Application program:

This is the general menu:

```
Book Store Main Menu
1. Log In
2. Sign up
3. View products
4. Make an Order
5. View your orders
6. Return an item
7. Quit
Enter choice:
```

First, you can create an account with option 2:

```
Enter choice: 2
    Enter first name: Bob
    Enter last name: Smith
    Enter email: bsmith@gmail.com
    Enter password: hi
    Account created successfully. You can now login.
```

Then, you can log in using an account you create or one already in the database, using option 1:

```
Enter choice: 1
    Please Log In (If you do not have an account, go sign up for a new account.):
        Enter email: bsmith@gmail.com
        Enter password: hi
    Log in successful. Welcome Bob Smith!

Book Store Main Menu
    (Logged in as Bob Smith)
1. Log Out
2. Sign up
3. View products
4. Make an Order
5. View your orders
6. Return an item
7. Quit
Enter choice:
```

You can also search products with option 3. You do not need to be logged in for this.
If you do a product search, a submenu opens up:

```
Enter choice: 3

    === VIEW PRODUCTS ===
    1. View all products
    2. Search product by name
    3. View toys
    4. View movies
    5. View books
    6. Back
    Choice:
```

Option 1 gives all the products. This screenshot was cut off since there are many products.
Option 2 lets you search by any keyword contained in the name.
Option 3 lets you see a list of all the toys. This screenshot was also cut off

```
Choice: 1

All Products:
ID: 1 | Wooden Train | $29.99
ID: 2 | Dragon Quest | $15.50
ID: 3 | Superhero Saga | $19.99
ID: 4 | Math Puzzle | $12.75
ID: 5 | Space Odyssey | $18.00
ID: 6 | Speed Racer | $49.99
ID: 7 | Candy Kingdom | $22.50
ID: 8 | Love in Paris | $14.25
ID: 9 | Nightmare Street | $17.99
ID: 10 | Pride & Prejudice | $12.50
ID: 11 | Cuddly Bear | $9.99
ID: 12 | Brain Twister | $11.75
```

```
Choice: 2
Enter product name keyword: math

Search Results:
ID: 4 | Math Puzzle | $12.75
ID: 19 | Math Made Easy | $14.50
```

```
Choice: 3

Toys:
ID: 1 | Wooden Train | $29.99
ID: 4 | Math Puzzle | $12.75
ID: 6 | Speed Racer | $49.99
ID: 7 | Candy Kingdom | $22.50
ID: 11 | Cuddly Bear | $9.99
ID: 12 | Brain Twister | $11.75
ID: 15 | Junior Scientist | $34.99
ID: 16 | Captain Bold | $24.50
```

Options 5 and 6 let you see all of the movies and books in a manner similar to option 4, but also allow you to sort by genre as shown:

```
Choice: 4

=== MOVIES ===
1. View all movies
2. View movies by genre
3. Back
Choice: 2

Choose a movie genre:
1. Animation
2. Documentary
3. Educational
4. Horror
5. Romance
6. SciFi
7. Back
Choice: 3

Movies in genre: Educational
ID: 13 | Flying Friends | $16.50
ID: 18 | Nature Wonders | $18.99
```

```
Choice: 5

=== BOOKS ===
1. View all books
2. View books by genre
3. Back
Choice: 2

Choose a book genre:
1. Educational
2. Fantasy
3. Horror
4. Mystery
5. Romance
6. SciFi
7. Back
Choice: 6

Books in genre: SciFi
ID: 5 | Space Odyssey | $18.00
```

Then back to the general menu, you can then make an order with option 4. You can add as many items as you want, but you need the product IDs which you can find using option 3 above and searching for what you want.

```
Enter choice: 4
    Do you want to add an item (1) or finalize and make your payment (2)?: 1
        Enter product id (you can search beforehand to get this): 5
        Enter quantity: 3
    Do you want to add an item (1) or finalize and make your payment (2)?: 1
        Enter product id (you can search beforehand to get this): 9
        Enter quantity: 1
    Do you want to add an item (1) or finalize and make your payment (2)?: 1
        Enter product id (you can search beforehand to get this): 1
        Enter quantity: 6
    Do you want to add an item (1) or finalize and make your payment (2)?: 2
        Total price: 251.93
        Enter payment method: Visa
        Enter credit card number: 34734785346
        Enter shipping address: default address 123
    Order Completed! (For reference, order_id = 27)
    You will get an email with payment and shipping details.
```

Also, if you are not logged in, you get this message since you can't make an order without an account. (This is also true for options 5 and 6 shown below since they are customer-specific.)

```
Enter choice: 4
Sorry, you need to be logged in to this. Go back to the menu and log in.
```

Then, you can use option 5 to see all of your orders. Note that I logged out of the account we just created and logged into an account already existing on our data base to see a more full order history.

```
Book Store Main Menu
    (Logged in as Paul Kuame)
1. Log Out
2. Sign up
3. View products
4. Make an Order
5. View your orders
6. Return an item
7. Quit
Enter choice: 5
    Here are all of your orders Paul Kuame:
        Order ID 3 from 2025-02-01
            Superhero Saga (product_id 3), Quantity bought: 1, Price Paid: 19.99
            Love in Paris (product_id 8), Quantity bought: 2, Price Paid: 33.50
        Order ID 18 from 2025-07-01
            Space Odyssey (product_id 5), Quantity bought: 1, Price Paid: 18.00
            Stuffed Bunny (product_id 28), Quantity bought: 1, Price Paid: 14.75
            Mini Drone (product_id 29), Quantity bought: 1, Price Paid: 94.99
        Order ID 21 from 2025-07-20
            Wooden Train (product_id 1), Quantity bought: 3, Price Paid: 89.97
            Pride & Prejudice (product_id 10), Quantity bought: 2, Price Paid: 25.00
            Robot Dog (product_id 21), Quantity bought: 1, Price Paid: 59.99
        Order ID 22 from 2026-03-24
            Space Odyssey (product_id 5), Quantity bought: 4, Price Paid: 72.00
        Order ID 23 from 2026-03-26
            Superhero Saga (product_id 3), Quantity bought: 2, Price Paid: 39.98
            Speed Racer (product_id 6), Quantity bought: 2, Price Paid: 99.98
        Order ID 24 from 2026-03-26
            Superhero Saga (product_id 3), Quantity bought: 3, Price Paid: 59.97
```

Then, with option 6, you can return an item. First, I will show some of the possible errors.
First of all, you cannot return an order that isn't yours. Right now we are still logged in as Paul Kuame, so order ID 19 is not ours from above.

```
Enter choice: 6
    You will need the order id and product id of what you want to return.
    For the numbers, view all your orders first.
    Enter the order id: 19
    This is not a valid order id for your orders.
```

Next, you cannot do anything more if you put an order that was more than 30 days ago. And we can see from option 5 that order 21 was from 2025-07-20.

```
Enter choice: 6
    You will need the order id and product id of what you want to return.
    For the numbers, view all your orders first.
    Enter the order id: 21
    It has been more than 30 days. I'm sorry. You cannot return this item.
```

Now, if you put an order that is yours, within 30 days, you can choose which products to return. If you put a product ID that was not in the order, you get an error. Then, if you put one that was in the order, you can select the quantity. If you choose a quantity more than is remaining, you will be told how many units are left so you can adjust.

```
Enter choice: 6
    You will need the order id and product id of what you want to return.
    For the numbers, view all your orders first.
    Enter the order id: 24
    Enter the product id: 1
    You did not buy this product in this order.
    Do you want to try another one? (Y/N): Y
    Enter the product id: 3
    Enter the quantity to return: 7
    Sorry, you only have 3 units of this product left that you can return.
    Do you want to adjust your number or go back to menu? (A/M): A
    Enter the quantity to return: 3
    Successfully initiated return! You will get an email about how to ship it to us.
```

And as we saw from option 5, in order 23, Paul Kuame bought 2 units of product 6. So, if you return 1 unit, then in the next iteration if you try to return 2 units, it will tell you you have only 1 unit left that you can return.

```
Enter choice: 6
    You will need the order id and product id of what you want to return.
    For the numbers, view all your orders first.
    Enter the order id: 23
    Enter the product id: 6
    Enter the quantity to return: 1
    Successfully initiated return! You will get an email about how to ship it to us.

Book Store Main Menu
    (Logged in as Paul Kuame)
1. Log Out
2. Sign up
3. View products
4. Make an Order
5. View your orders
6. Return an item
7. Quit
Enter choice: 6
    You will need the order id and product id of what you want to return.
    For the numbers, view all your orders first.
    Enter the order id: 23
    Enter the product id: 6
    Enter the quantity to return: 2
    Sorry, you only have 1 units of this product left that you can return.
    Do you want to adjust your number or go back to menu? (A/M): A
    Enter the quantity to return: 1
    Successfully initiated return! You will get an email about how to ship it to us.
```

Indexing:

a)

```
db2 => CREATE INDEX index_oi_product ON OrderItem(product_id);
DB20000I  The SQL command completed successfully.
```

b) The index **index_oi_product** is created on the relation **OrderItem** and its attribute **product_id**. This is valid as the primary key of OrderItem is (order_id, product_id), so an

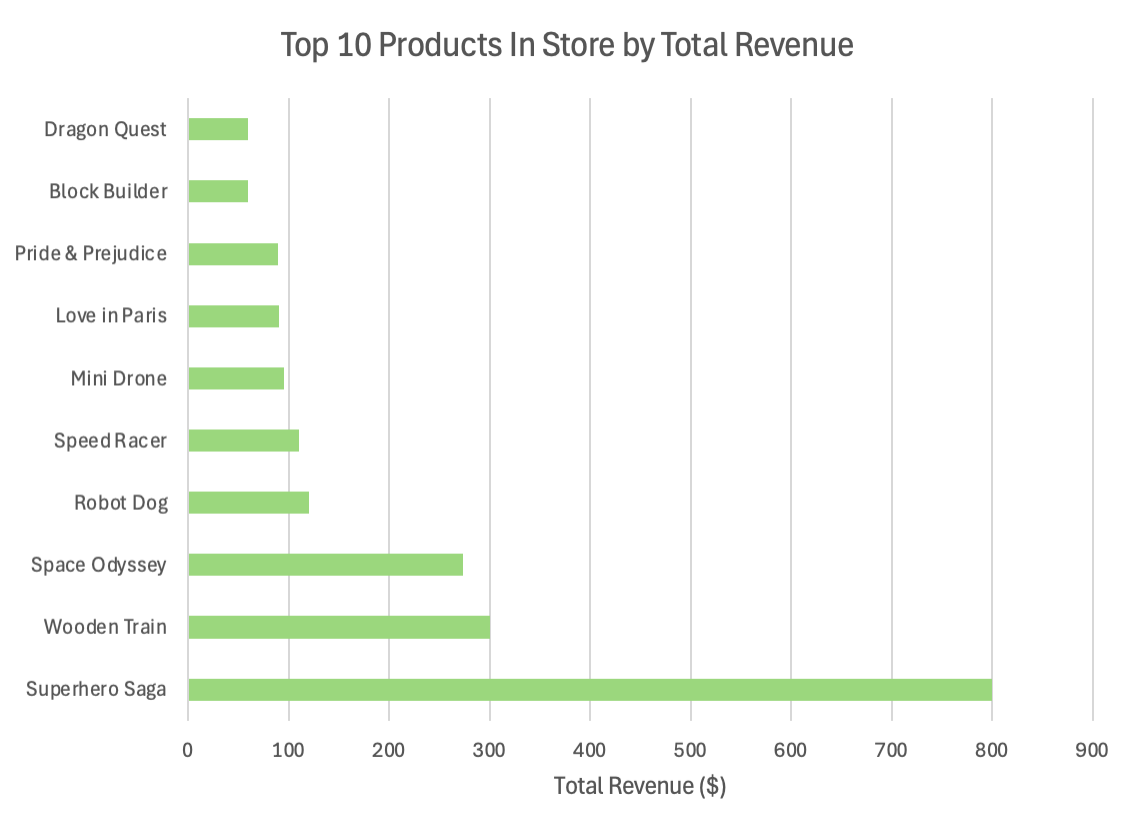
index isn't automatically created for product_id. This index speeds up queries that use OrderItem, such as those that look up order items by product. This includes any queries in the application that join OrderItem and Product on product_id. For example, Option 5 in the general menu (View your orders) and Option 6 (return an item) both query OrderItem filtering or joining on product_id. With this index, the application can jump directly to relevant rows instead of scanning every row in OrderItem for matches. In a real store, a new row is added for every item in every order, this index would be very useful as OrderItem grows larger.

Visualization:

a) SQL Statement:

```
EXPORT TO top10_revenue.csv OF DEL MODIFIED BY NOCHARDEL
SELECT p.name, SUM(oi.quantity * oi.price_at_purchase) AS total_revenue
FROM Product p JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY total_revenue DESC
FETCH FIRST 10 ROWS ONLY;
```

b) Visualization:



c) This graph presents the top 10 products that have generated the most revenue in the store. The first is "Superhero Saga", then "Wooden Train", and so on. It's simply ordered by the amount of revenue in dollars.

Creativity:

a) We chose to make a trigger for our application, since we had really wanted to in project 2. Our main remaining constraint was that you can only return an item within 30 days of the time of the order. However, since this was a constraint across tables, we couldn't verify it with a Check constraint. So, we created this trigger, which raises a custom error if you try to insert a return that isn't within 30 days of its order.

b) Code for the trigger:

```
CREATE TRIGGER returns_within_thirty_days
BEFORE INSERT ON "Return"
REFERENCING NEW AS NEW
FOR EACH ROW
BEGIN ATOMIC
    DECLARE original_order_timestamp TIMESTAMP;
    SET original_order_timestamp = (SELECT order_timestamp FROM "Order"
                                    WHERE order_id = NEW.order_id);

    IF (original_order_timestamp IS NOT NULL) AND
        (DAYS(NEW.return_timestamp) - DAYS(original_order_timestamp) > 30)
        THEN
            SIGNAL SQLSTATE '70001'
            SET MESSAGE_TEXT = 'Return date must be within 30 days of order date';
    END IF;
END @
```

c) The execution of the trigger:

```
cs421g27@winter2026-comp421:~$ db2 -td@ -vf project3/project3_trigger.sql
CREATE TRIGGER returns_within_thirty_days
BEFORE INSERT ON "Return"
REFERENCING NEW AS NEW
FOR EACH ROW
BEGIN ATOMIC
    DECLARE original_order_timestamp TIMESTAMP;
    SET original_order_timestamp = (SELECT order_timestamp FROM "Order" WHERE order_id = NEW.order_id);

    IF (original_order_timestamp IS NOT NULL) AND
        (DAYS(NEW.return_timestamp) - DAYS(original_order_timestamp) > 30)
        THEN
            SIGNAL SQLSTATE '70001'
            SET MESSAGE_TEXT = 'Return date must be within 30 days of order date';
    END IF;
END
DB20000I  The SQL command completed successfully.
```

d) First, as we can see, order 1 was made on Jan 10, 2025:

```
db2 => select * from "Order"
db2 (cont.) => where order_id = 1;

ORDER_ID    ORDER_TIMESTAMP            CUSTOMER_ID PAYMENT_ID  SHIPPING_ID
----------- -------------------------- ----------- ----------- -----------
          1 2025-01-10-10.00.00.000000           1           1           1

  1 record(s) selected.
```

So, if we try to insert a return for order 1 on Mar 10, 2025, we get the error as we wanted:

```
db2 => insert into "Return" (return_timestamp, order_id, product_id, quantity, shipping_id)
db2 (cont.) => values ('2025-03-10 14:00:00',1,2,1,3);
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0438N  Application raised error or warning with diagnostic text: "Return
date must be within 30 days of order date".  SQLSTATE=70001
```

But, if we insert a return for order 1 on Jan 20, 2025, everything works correctly:

```
db2 => insert into "Return" (return_timestamp, order_id, product_id, quantity, shipping_id)
db2 (cont.) => values ('2025-01-20 03:00:00',1,2,1,3);
DB20000I  The SQL command completed successfully.
```

Member Contributions:

We had one meeting to discuss how we were going to split the project and then discussed further through text. Steven did Task 3 and set up Task 4. Alice and Mujtaba did half of Task 4 each. Eleni did Tasks 5 and 6. Alice did Task 7.